



LoRA 微调：从原理到调参

AI 大模型时代的“轻”功秘籍

本次分享大纲：从“为什么”到“是什么”再到“怎么做”



1. 引言：大模型微调的“重”与“痛”

传统全量微调的挑战



2. 破局：LoRA 的核心思想与优势

给大模型动个“小手术”



3. 原理：LoRA 如何化繁为简？

低秩近似的数学魔法



4. 实战：LoRA 核心参数与调优技巧

从入门到精通

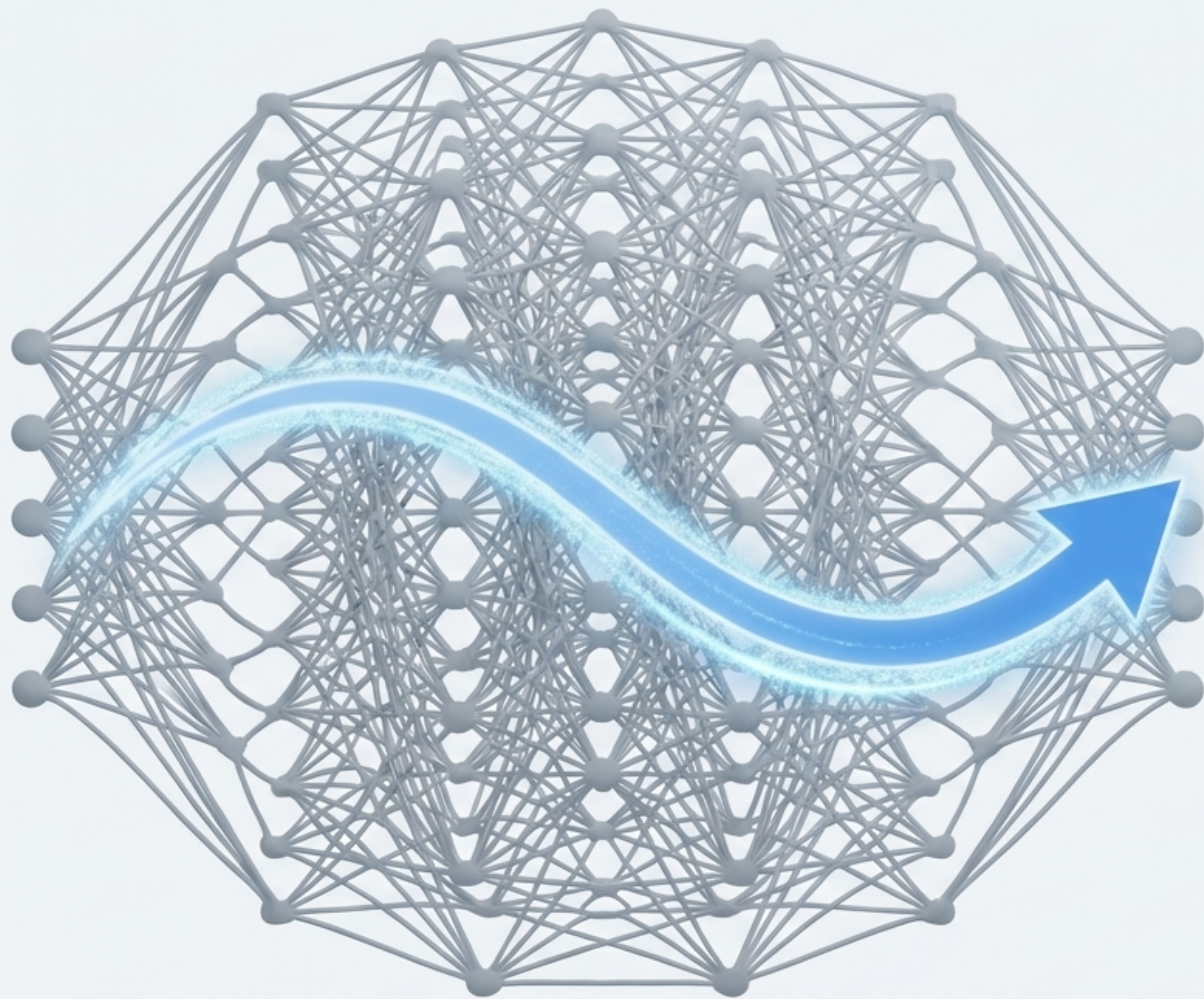


5. 演进：LoRA 的变体与未来

LoRA 家族生态



6. 总结：LoRA 的价值与展望



传统微调：重写整本“百科全书”

方式：全量微调（Full Fine-tuning）

定义：在新任务上训练时，更新模型底层的**所有**权重参数
($W' = W + \Delta W$)。

类比：假设你有一本包含 100 亿知识点的“百科全书”（预训练模型）。为了教它一个新技能，你需要把整本书从头到尾重写一遍。

全量微调的“两大痛点”：资源与管理的噩梦



计算资源消耗巨大

显存噩梦：训练一个百亿级模型需要昂贵的计算集群。时间周期长，模型迭代和实验效率极低。



存储与部署困难

存储噩梦：每个微调任务都需要保存一个完整的模型副本。例如，一个 10B 模型的 ``float32`` 权重就接近 **40GB**。几十个任务意味着致数百 GB 的存储和复杂的版本管理。

LoRA 破局：只为“百科全书”贴一张“补充说明”



- **冻结 + 旁路：**冻结预训练模型的全部参数，保持其强大的泛化能力。
- **注入新知识：**在模型的特定层旁边，注入一个可训练的、极小的“旁路网络”（LoRA模块）。
- **微调即训练旁路：**整个微调过程，只训练这个小小的 LoRA 模块，从而避免了灾难性遗忘。

LoRA 的三大核心优势



省 (Economical)

可训练参数量锐减 **99%+**：将需要训练的参数从亿级降至百万甚至万级。例如，对 GPT-3 (175B) 进行 LoRA 微调，可训练参数仅占总量的 **0.01%**。



快 (Fast)

训练速度显著提升：由于需要计算梯度的参数极少，训练效率大幅提高。任务切换成本为零，快速加载不同 LoRA 文件即可。



灵 (Flexible)

模块化即插即用：LoRA 模块仅几 MB，便于存储、分享和部署。多个团队可以基于同一基础模型，并行开发不同的 LoRA 模块。

核心原理：权重更新的低秩近似

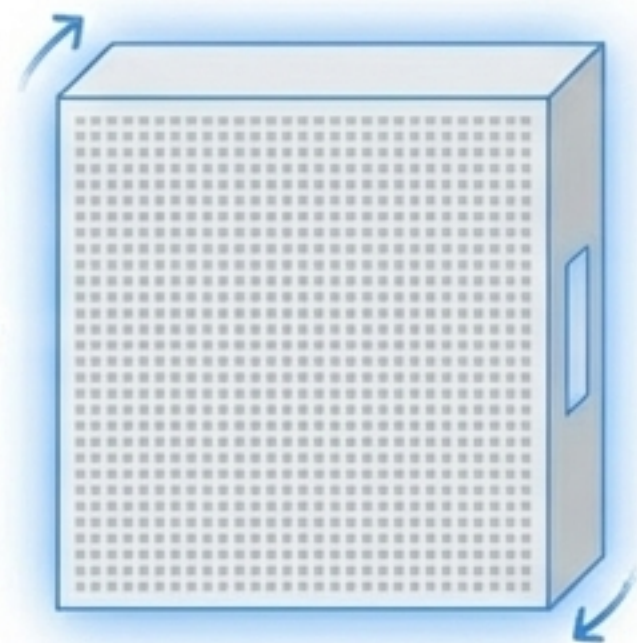
模型微调的本质

$$W_{\text{new}} = W_{\text{old}} + \Delta W$$



W_{old}
庞大且固定的
预训练权重

+



ΔW
需要学习的、同样
庞大的权重变化量



LoRA 的核心洞察 (Hypothesis)

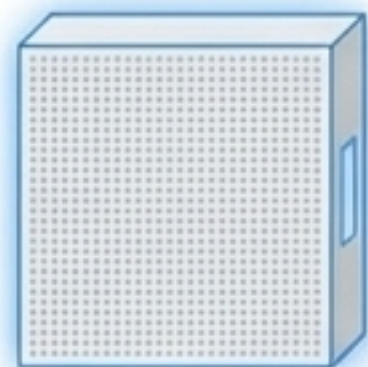
模型在适应新任务时，其权重变化量 ΔW 具有“内在的低秩结构” (low intrinsic rank)。

这意味着 ΔW 这个大矩阵可以用两个更小的矩阵 B 和 A 的乘积来高度近似。



LoRA 的实现

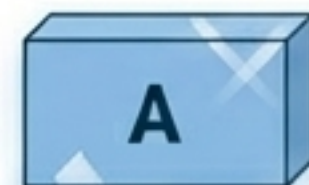
$$\Delta W \approx B \times A$$



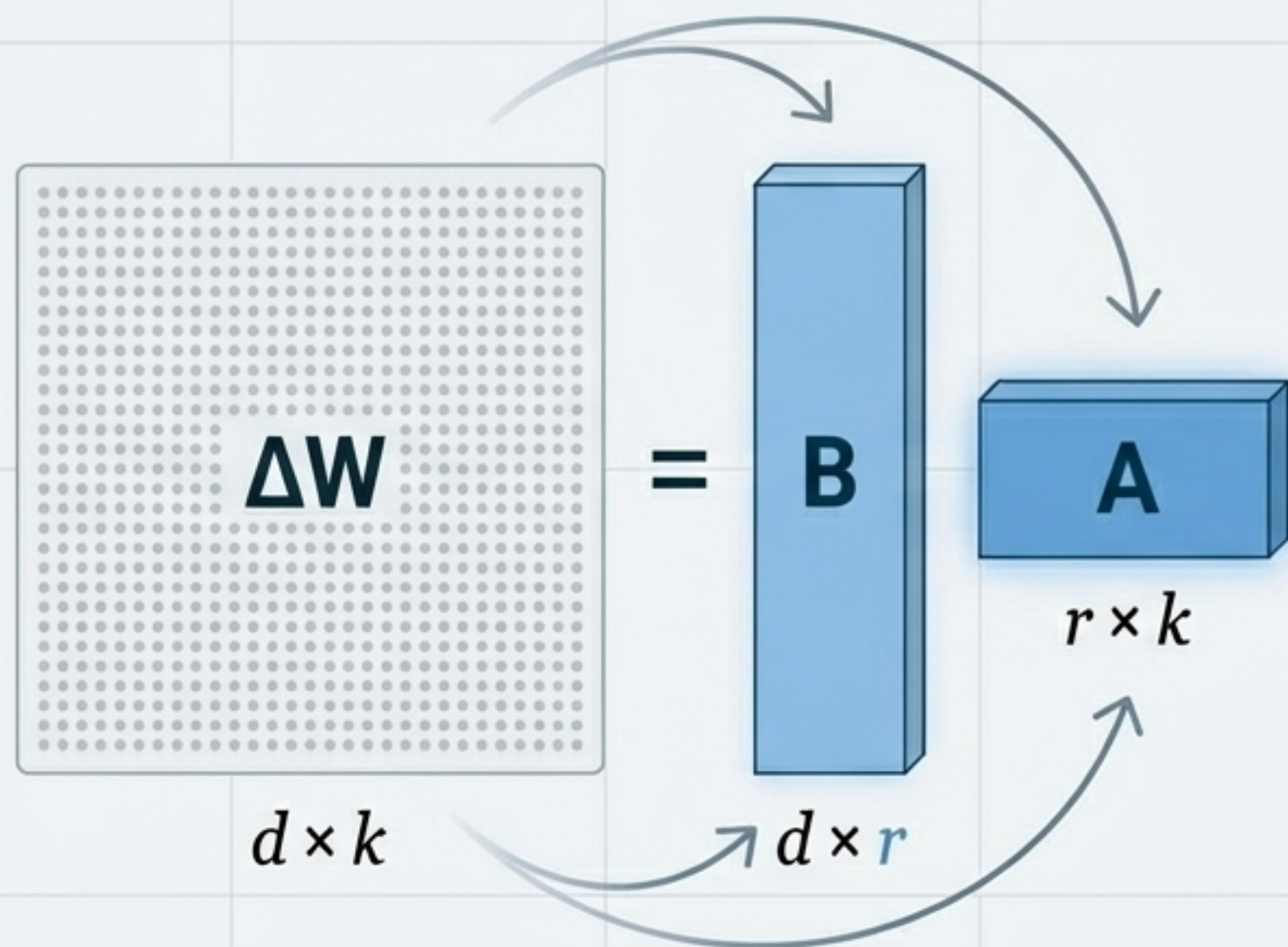
≈



×



参数量如何实现万倍压缩？



参数量对比示例（以 $d=4096$, $k=4096$ 为例）：

全量微调 ΔW : $4096 \times 4096 = 16,777,216$ 个参数。

使用 LoRA（设 $r=8$ ）：

- 矩阵 A ($r \times k$): $8 \times 4096 = 32,768$
- 矩阵 B ($d \times r$): $4096 \times 8 = 32,768$
- 总参数量: $32,768 + 32,768 = 65,536$

压缩比: $65,536 / 16,777,216 \approx 0.39\%$ 。参数量降至千分之四！

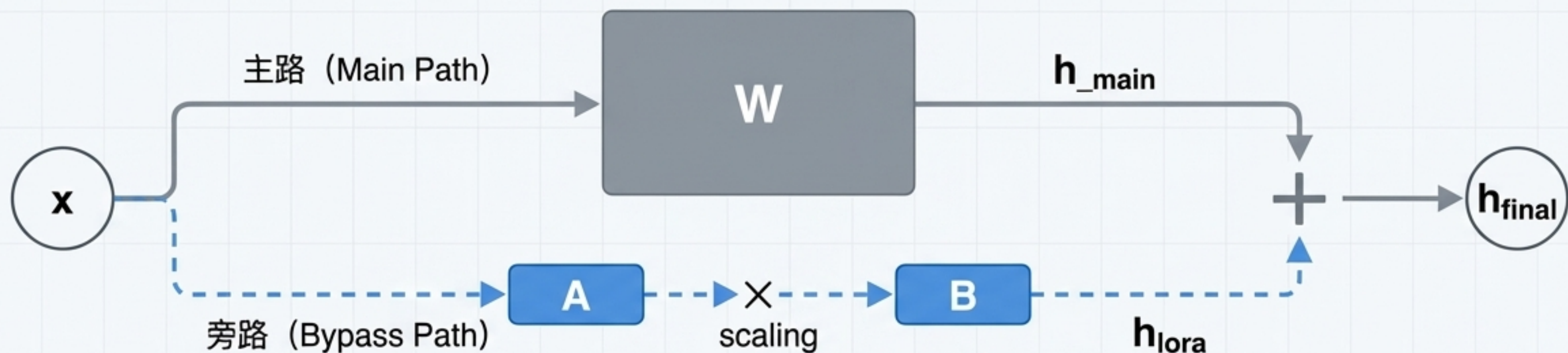
理论支撑：奇异值分解（SVD）的启示



核心思想：

1. SVD 证明，任何矩阵的“信息量”都集中在最大的少数几个奇异值上。
2. 丢弃小的奇异值，只保留最大的 r 个，就能以很小的精度损失，极大地压缩矩阵。
3. 实例证明：在一个矩阵 SVD 实验中，仅保留最大的 3 个奇异值就能保留 **99.97% 的信息量**，重建后的矩阵与原始矩阵高度相似。
 $\frac{(7.03+3.00+2.15)}{63.06} \approx 99.97\%$
4. 这为“ ΔW 可以被低秩矩阵 $B \cdot A$ 高效近似”提供了坚实的数学理论依据。

LoRA 的前向传播过程



$$h_{\text{final}} = W \cdot x + (B \cdot A) \cdot x$$

****训练循环关键步骤（五步法）：**

1. **初始化：**冻结 W 。 A 随机高斯初始化， B 初始化为零（确保训练开始时 $\Delta W=0$ ）。
2. **前向传播：**计算 h_{final} 并得出预测结果。
3. **计算损失：**根据预测与真实标签计算损失函数 L 。
4. **反向传播：**关键：梯度只回传到 A 和 B 。 W 完全不参与梯度计算。
5. **参数更新：**优化器（如 AdamW）根据梯度更新 A 和 B 的权重。

核心超参数配置指南

```
peft_config:
  peft_type: "LORA"
  r: 16      # 秩 (Rank) , 决定了LoRA模块的表达能力
  lora_alpha: 32    # 缩放因子 (Scaling Factor)
  lora_dropout: 0.1 # Dropout概率, 防止过拟合
  target_modules:   # 要应用LoRA的模块名
    - "q_proj"
    - "v_proj"
```

**经验法则:

- **r (Rank):** 最关键的参数。决定了近似矩阵的“容量”，即能够学习多复杂的信息。
- **lora_alpha:** 缩放系数。经典设置是 r 的两倍 ($\alpha = 2 * r$)，与学习率共同调节学习强度。
- **target_modules:** 通常首先选择注意力机制中的 `q_proj` (查询) 和 `v_proj` (值)。

成为 LoRA 调参高手：实战技巧



Tip 1: 从小 `r` 开始验证

先用 `r=8` 或 `r=16` 快速跑通训练流程，验证数据和代码正确性，再逐步增加 `r` 探索性能上限。



Tip 2: 数据量决定 `r` 的上限

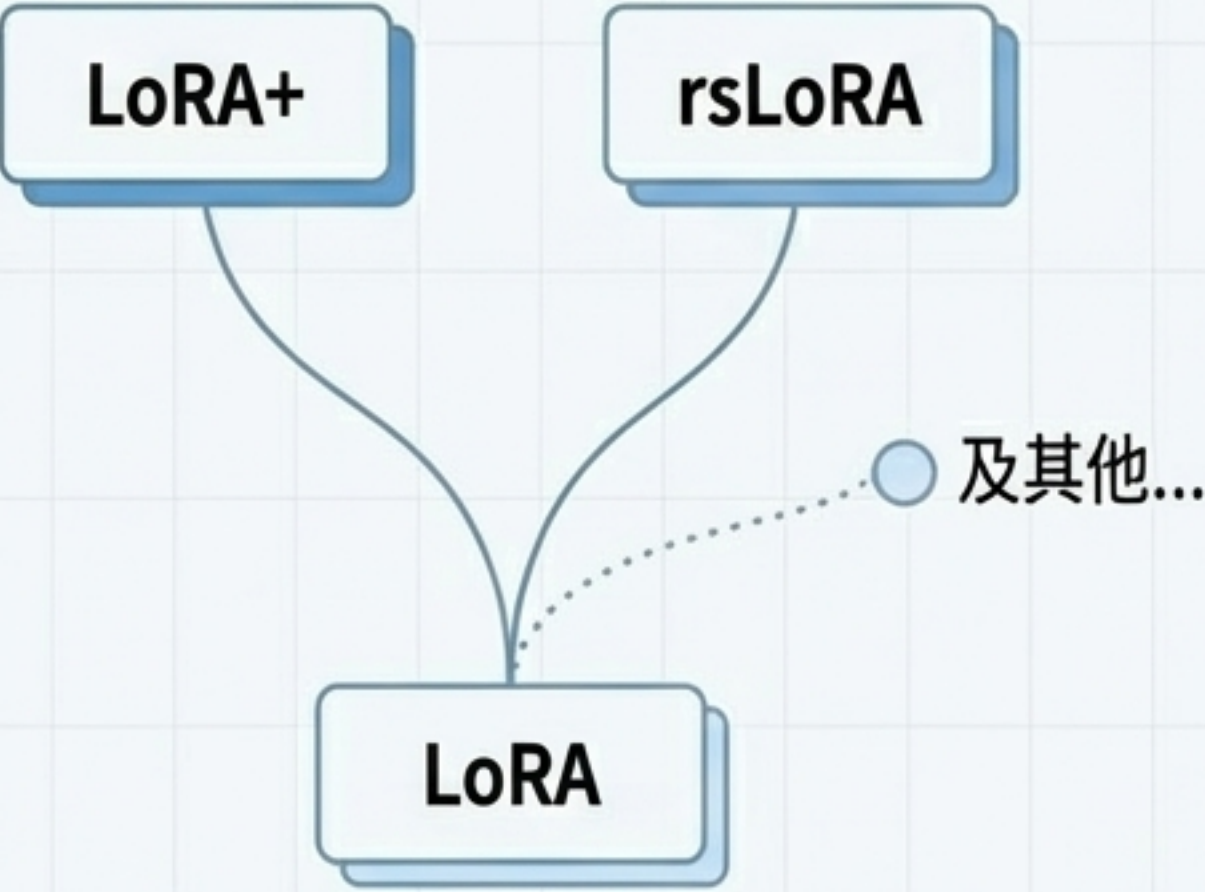
小数据集 (< 5k 样本)： 建议使用小 `r` (例如 `r=8`)，大 `r` 极易过拟合。
大数据集或复杂任务： 可尝试更大的 `r` (例如 `r=32` 或 `r=64`)。



Tip 3: 扩展 Target Modules

如果任务复杂，除了 `q\_proj` 和 `v\_proj`，可以考虑增加 `k\_proj`，`o\_proj` 甚至 `FFN` 层的线上的线性模块，以提升模型容量。

LoRA 永不眠：生态演进



变体	核心改进	优势
LoRA+	为矩阵 `A` 和 `B` 设置不同的学习率（例如 `B` 的学习率远高于 `A`）。	理论上更稳定，收敛效果更好，性能可提升 1%-2%。
rsLoRA (Rank-Stabilized LoRA)	改变 `alpha` 的设置方式，使其与 `r` 解耦（alpha 固定，缩放系数变为 ` $\alpha/\sqrt{r}$ `）。	对超参 `r` 的选择不再敏感，简化调参，更加灵活。
及其他...	DoRA, PISSA, VeRA, LoKr 等持续探索更高效的低秩近似方法。	性能、效率、稳定性的不断提升。

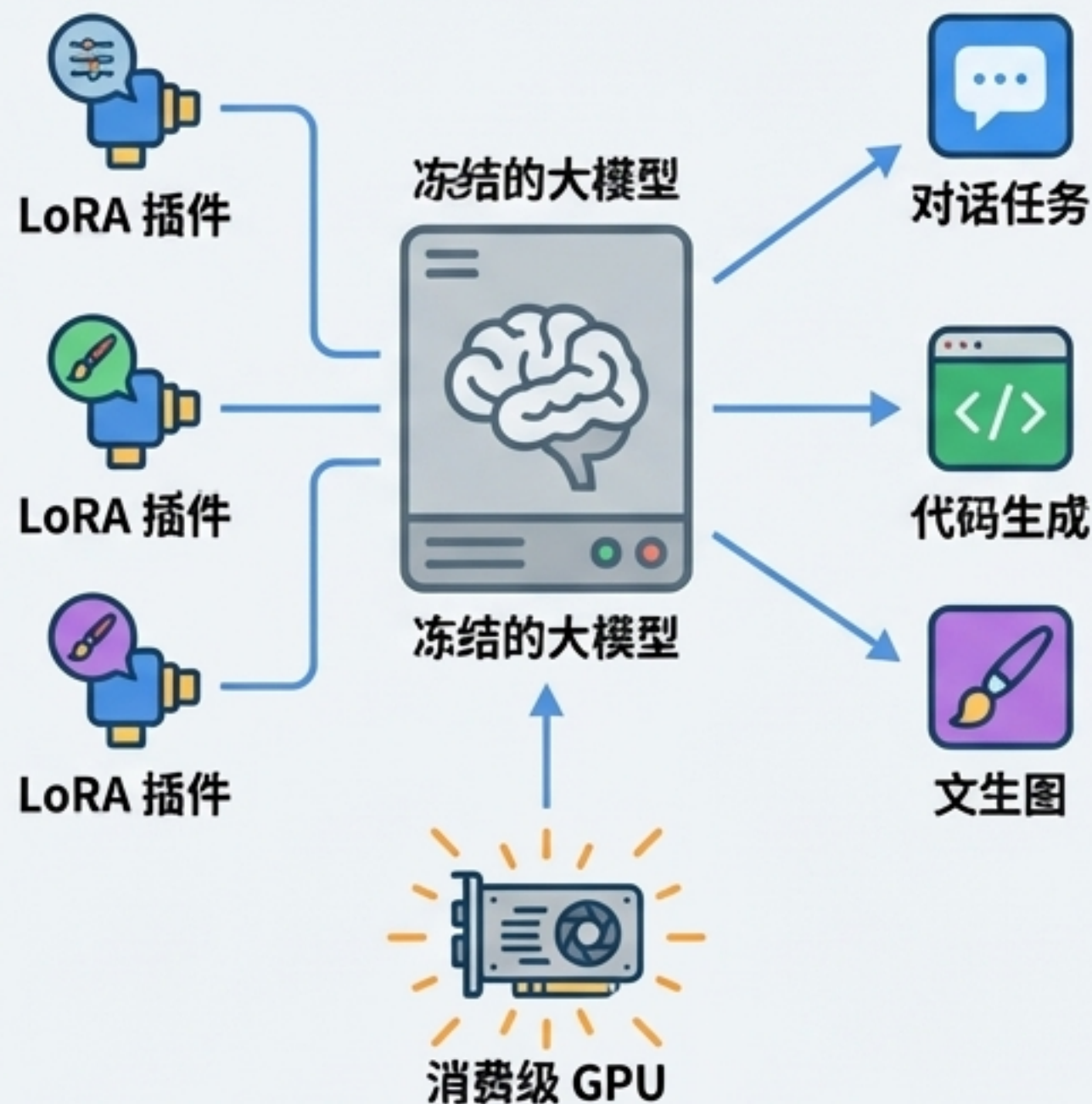
总结：LoRA，大模型微调的民主化革命

原理精髓：

- 通过**低秩分解**，用极小的代价（ B 和 A ）高效近似大模型的权重更新（ ΔW ）。其背后有**SVD**理论的坚实支撑。

实践准则：

- r 是容量， α 是强度。从 $r=16$, $\alpha=32$ 开始，根据任务复杂度和数据量进行调整。



核心价值：

- 将大模型微调的门槛从“工业级”拉到“个人级”，**democratizing** AI 技术。它实现了参数、训练、部署的全方位高效。

未来展望：

- LoRA 是参数高效微调（PEFT）的基石。更智能、自适应的 LoRA 变体将使微调过程更加自动化和高效。